502094 - Software Deployment, Operations And Maintenance

Module 14: DevSecOps: Integrating Security in CI/CD Pipelines

Lecturer: MSc. Mai Van Manh

Email: maivanmanh@tdtu.edu.vn

Core DevSecOps Concepts

- Why DevSecOps matters for modern backend systems
- How security is integrated into CI/CD pipelines
- Key ideas: **shift left**, **continuous security**, **shared responsibility**

What Is DevSecOps?

- **DevSecOps** = Development + Security + Operations
- Extends DevOps by embedding security practices into every stage of the SDLC
- Goal: **build secure software by default**, not bolt-on security at the end
- Security becomes a **shared responsibility** across developers, ops, and security teams

Why DevSecOps for Backend & DevOps?

- Backend services handle **data**, **authentication**, **business logic** → high-value targets
- CI/CD pipelines can **deploy insecure code at high speed** if not protected
- Cloud & containers increase **attack surface** and complexity
- DevSecOps helps:
  - Catch vulnerabilities **earlier**
  - Reduce production incidents
  - Improve compliance and audit readiness

The "Shift Left" Philosophy

- Traditional model:
  - Code → Build → Test → **Security review** → Deploy
- Problems:
  - Security discovered **late**
  - Expensive to fix after release
  - Security becomes a **bottleneck**
- **Shift left**: move security checks as early as possible into:
  - Coding
  - Build & test
  - Pull requests

Shift Left: Timeline View

- Typical secure pipeline:
  - **Code**: secure coding guidelines, secret detection, SAST
  - **Build**: dependency scanning (SCA), image scanning
  - **Test**: DAST, penetration tests for critical flows
  - **Deploy**: policy checks, runtime protections
- Earlier detection → cheaper and faster fixes

Shift Left Example: Pull Request Checks

- Developer opens PR for a Node.js backend
- CI pipeline automatically runs:
  - Unit tests
  - **Static analysis** to catch insecure code patterns
  - **Dependency scanning** on `package.json`
- PR is blocked if:
  - Tests fail
  - High-severity vulnerabilities are found

Shift Left Example: Pull Request Checks

```yaml
# .github/workflows/pr-checks.yml (excerpt)
jobs:
  security-checks:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
      - run: npm ci
      - run: npm audit --audit-level=high
```

Common Attack Vectors in Modern Web Apps

- **Injection attacks** (SQL, NoSQL, OS command injection)
- **Cross-Site Scripting (XSS)** and cross-site request forgery (CSRF)
- **Authentication & session** weaknesses
- **Broken access control** (users seeing or modifying others' data)
- **Insecure dependencies** with known CVEs
- **Misconfigured infrastructure** (open ports, public S3 buckets)

Attack Vectors in Containerized Environments

- Running containers as **root** inside the container
- Over-permissive **host mounts** (e.g., mounting `/var/run/docker.sock`)
- Unpatched base images with **old OS packages**
- Wide-open **ingress** and overly permissive **network policies**
- Over-privileged **Kubernetes ServiceAccounts** or CI/CD IAM roles

Example: Dependency Vulnerability

- Backend uses an old JWT library with a critical vulnerability
- No one updates it for months
- Attacker crafts a malicious token exploiting the bug → bypasses login
- Without DevSecOps:
  - Vulnerability is discovered only after an incident
- With DevSecOps:
  - Dependency scanner flags it shortly after it is published
  - Build fails until the dependency is upgraded

Security Integration Points in CI/CD

1. **Code & Commit Stage**
   - Secret scanning, SAST, pre-commit hooks
2. **Build & Test Stage**
   - Dependency scanning (SCA)
   - Container image scanning
   - Unit and integration tests
3. **Deploy & Runtime Stage**
   - IaC and configuration scanning
   - Policy-as-code gates in CD, Runtime monitoring and alerts

Mapping Security Checks to Pipeline Stages

```
[ Developer Laptop ]
    |
    v
[ Git Repository ]
    - Pre-commit hooks
    - Secret scanning bot
    |
    v
[ CI Build ]
    - SCA (dependencies), SAST
    - Container scan
    |
    v
[ CD Deploy ]: IaC scan, OPA policies
    |
    v
[ Production Cluster ]
    - Monitoring & logging
    - Alerting & incident response
```

Practical Security Scanning

- Focus on **automated** security checks inside CI
- Three key scanning types:
  - **Dependency scanning** (SCA)
  - **Container image scanning**
  - **Configuration / IaC scanning**
- Goal: make insecure builds **fail fast**

Dependency Scanning (SCA) – Concepts

- **SCA (Software Composition Analysis)**:
  - Analyses open-source dependencies
  - Matches them against **known vulnerabilities** databases (e.g. NVD)
- Why it matters:
  - Modern apps are mostly **dependencies + glue code**
  - A single vulnerable library can compromise the whole system
- Typical tools:
  - `npm audit`, `yarn audit`, Snyk, GitHub Dependabot

SCA Scenario: Node.js Backend

- Project structure:

```
my-api/
  package.json
  package-lock.json
  src/
    index.js
```

- SCA checks:
  - Which packages are used?
  - Are there known CVEs for these versions?
  - Are there fixes available?
- SCA is run on **every build** and **every PR** to prevent regressions

Running `npm audit` Locally

```
cd my-api
npm install
npm audit --audit-level=high
```

- `npm audit`:
  - Checks dependencies against a vulnerability database
  - Returns severity levels: low, moderate, high, critical
  - Use `--audit-level=high` to fail only on high/critical issues
- Good habit: developers run this **before pushing** their code

Integrating SCA into GitHub Actions

```yaml
name: Security - Dependency Scan
on:
  pull_request:
    branches: [ main ]
jobs:
  npm-audit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
      - run: npm ci
      - name: Run npm audit
        run: npm audit --audit-level=high
```

- If `vulnerabilities ≥ high severity` are found: The job fails; PR cannot be merged until issues are fixed

Using Snyk for Deeper SCA

```
# One-time: authenticate
snyk auth

# In project
cd my-api
snyk test --file=package.json
```

- Snyk provides:
  - More detailed remediation advice
  - PRs to automatically bump versions
- Can be integrated as a CI step similar to `npm audit`

Interpreting SCA Results

Typical output:

```
High: Prototype Pollution in lodash
- Introduced through: my-api@1.0.0
- Vulnerable module: lodash
- Fixed in: 4.17.21

Remediation:
- Run `npm install lodash@4.17.21`
```

- Developer actions:
  - Update the dependency
  - Re-run tests to ensure nothing breaks
  - Re-run SCA to confirm vulnerability is resolved

Container Image Scanning – Concepts

- Even if application dependencies are safe, the **base image** may be vulnerable
- Container scanners check:
  - OS packages (e.g., `openssl`, `glibc`)
  - Language runtimes
  - Installed tools
- Tools: **Trivy**, Grype, Clair, etc.
- Goal: avoid shipping images with known OS-level vulnerabilities

Trivy Basics

- Install Trivy locally (Linux)

```
sudo apt update
sudo apt install -y wget
wget https://github.com/.../trivy_Linux-64bit.tar.gz
tar -xzf trivy_Linux-64bit.tar.gz
sudo mv trivy /usr/local/bin/
```

- Scan a local image:

```
trivy image my-api:latest
```

- Output lists vulnerabilities with severity and recommended fixes

Trivy CI Integration (GitHub Actions)

```yaml
name: Security - Container Scan
on:
  push:
    branches: [ main ]
jobs:
  trivy-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build Docker image
        run: docker build -t my-api:ci .
      - name: Run Trivy scan
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: 'my-api:ci'
          exit-code: '1' # fail build if high/critical issues are found
          severity: 'HIGH,CRITICAL'
```

Interpreting Trivy Output

Example snippet:

```
ubuntu 22.04 (jammy)
Package      Vulnerability  Severity  Installed   Fixed
------------ -------------- --------- ----------- ------------
openssl      CVE-2023-1234 HIGH      3.0.2-0     3.0.2-1

Suggestions:
- Rebuild the image using a newer base image tag
- Or explicitly update the package in Dockerfile
```

- Action:
  - Choose a newer base image (e.g., `node:20-alpine`)
  - Rebuild and rescan

Configuration / IaC Scanning – Concepts

- Infrastructure as Code (IaC) defines:
  - VPCs, subnets, security groups (Terraform)
  - Deployments, Services, Ingress (Kubernetes manifests)
- Misconfigurations can introduce severe risk:
  - Public security groups
  - World-readable S3 buckets
  - Privileged containers
- IaC scanning tools (e.g., **Checkov**) detect issues **before** deployment

IaC Scanning with Checkov

- Install Checkov:

```
pip install checkov
```

- Run against Terraform code:

```
cd infra/terraform
checkov -d .
```

- Checkov inspects:
  - Security groups, IAM roles and policies
  - Storage configurations
  - Kubernetes resources (if used)

Example Checkov Finding

```
Check: CKV_AWS_23
Resource: aws_security_group.web_sg

Problem: Security group allows ingress from 0.0.0.0/0 on port 22 (SSH)

Fix:
- Restrict CIDR range to trusted IPs only
- Or use a bastion host / VPN
```

- DevSecOps practice:
  - Fix IaC, Commit change
  - Re-run Checkov
  - Only deploy when checks pass

Secrets Management

- One of the **most common** security mistakes:
  - Hardcoding secrets in code or config files
- Risks:
  - Secrets leak via Git repository, screenshots, or logs
  - Anyone with access to repo can see database passwords or API keys
- DevSecOps approach:
  - Use **secret managers**
  - Use **CI/CD secret variables**
  - Automate secret injection at deploy time

Problem with Hardcoded Secrets

Bad example:

```
// config.js
module.exports = {
  db: {
    host: 'db.example.internal',
    user: 'myapp',
    password: 'SuperSecret123!'
  }
};
```

- Problems:
  - Password is visible to every developer with repo access
  - If repo is leaked, password is exposed
  - Harder to rotate credentials

Problem with .env Files Without Care

Typical setup:

```
DB_HOST=db.example.internal
DB_USER=myapp
DB_PASSWORD=SuperSecret123!
```

- Risks:
  - `.env` accidentally committed to Git
  - Shared via email or chat
- Minimum practice:
  - Add `.env` to `.gitignore`
  - Use environment-specific `.env` files that never leave the server

Secure Secret Injection: Using Environment Variables

Better application code:

```
// config.js
module.exports = {
  db: {
    host: process.env.DB_HOST,
    user: process.env.DB_USER,
    password: process.env.DB_PASSWORD
  }
};
```

- CI/CD pipeline:
  - Reads secrets from a **secret manager** or CI variable store
  - Injects them as environment variables when deploying

Secrets in GitHub Actions

- Store secrets in repository settings: **Settings → Secrets and variables → Actions**
- Example deployment job:

```yaml
jobs:
  deploy:
    runs-on: ubuntu-latest
    env:
      DB_HOST: ${{ secrets.DB_HOST }}
      DB_USER: ${{ secrets.DB_USER }}
      DB_PASSWORD: ${{ secrets.DB_PASSWORD }}
    steps:
      - uses: actions/checkout@v4
      - name: Deploy application
        run: ./deploy.sh
```

- Secrets never appear in the repository; GitHub masks them in logs

Using AWS Secrets Manager (Example)

- Store secret in AWS Secrets Manager, e.g. `prod/db-credentials`
- CI job fetches the secret:

```
aws secretsmanager get-secret-value \
  --secret-id prod/db-credentials \
  --query SecretString \
  --output text > db-secret.json
```

- Then export environment variables in the deploy script:

```
export DB_USER=$(jq -r '.username' db-secret.json)
export DB_PASSWORD=$(jq -r '.password' db-secret.json)
```

- Application uses `process.env` as shown before

IAM Hardening – Principle of Least Privilege

- **Principle of Least Privilege (PoLP)**:
  - Every user, service, or role should have **only** the permissions it needs
- For CI/CD:
  - The pipeline role should deploy the app, not manage the entire account
- Benefits:
  - Limit blast radius if credentials are compromised
  - Easier to reason about who can do what

Example: Over-Privileged IAM Policy (Bad)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "*",
      "Resource": "*"
    }
  ]
}
```

- CI role with this policy can:
  - Delete databases, Manage IAM users
  - Create and delete any resource
- A single compromise → full account takeover

Example: Least-Privilege IAM Policy (Better)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ecs:UpdateService",
        "ecs:DescribeServices",
        "ecs:DescribeTaskDefinition",
        "ecs:RegisterTaskDefinition"
      ],
      "Resource": "*"
    }
  ]
}
```

Example: Least-Privilege IAM Policy (Better)

- CI role can:
  - Register new ECS task definitions
  - Update ECS services
- It cannot:
  - Manage IAM users
  - Delete RDS databases
  - Modify unrelated infrastructure

Reviewing CI/CD Permissions

- Steps to harden CI role:

1. List what the pipeline actually does (e.g., deploy to ECS, update S3).
2. Create an IAM policy containing **only** those actions.
3. Attach the policy to the CI role and remove `*:*` permissions.
4. Periodically review logs:
    - CloudTrail / audit logs
    - Check for denied actions that indicate missing permissions

Policy Enforcement & Review

- Having policies is not enough; they must be **enforced**
- Policy-as-Code (PaC):
  - Write policies in a machine-readable language
  - Store them in version control
  - Enforce them automatically in CI/CD
- Common tools:
  - **Open Policy Agent (OPA)**
  - OPA Gatekeeper for Kubernetes
  - Conftest for local testing

Policy-as-Code: Key Ideas

- Policies define **what is allowed** or **denied**
- Examples:
  - All images must come from a trusted registry
  - No container may run as root
  - All Ingress resources must use HTTPS
- Benefits:
  - Repeatable and consistent across environments
  - Can be reviewed, tested, and versioned like code

OPA Gatekeeper Overview

- Gatekeeper is a Kubernetes admission controller
- It uses:
  - **ConstraintTemplates**: define reusable policy logic
  - **Constraints**: apply policies to specific resources
- When a resource is created/updated:
  - Gatekeeper evaluates it against the policies
  - If it violates a policy, the request is rejected

Example Policy: No Containers Running as Root

```yaml
apiVersion: templates.gatekeeper.sh/v1beta1
kind: ConstraintTemplate
metadata:
  name: k8srequirednonroot
spec:
  crd:
    spec:
      names:
        kind: K8sRequiredNonRoot
  targets:
    - target: admission.k8s.gatekeeper.sh
      rego: |
        package k8srequirednonroot

        violation[{"msg": msg}] {
          container := input.review.object.spec.containers[_]
          not container.securityContext.runAsNonRoot
          msg := "Containers must set securityContext.runAsNonRoot = true"
        }
```

Example Constraint Applying the Policy

```yaml
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sRequiredNonRoot
metadata:
  name: disallow-root-containers
spec:
  match:
    kinds:
      - apiGroups: [""]
        kinds: ["Pod"]
```

- If a Pod definition does not set `runAsNonRoot: true`, Gatekeeper rejects it
- This prevents developers from accidentally running containers as root

Gatekeeper in a Deployment Workflow

- Typical flow:

1. Developer applies Kubernetes YAML (`kubectl apply -f deployment.yml`).
2. Kubernetes API server forwards request to Gatekeeper.
3. Gatekeeper checks constraints:
    - Non-root container?
    - Trusted image registry?
4. If violation:
    - Request is rejected
    - Developer sees a clear error message

- Security is enforced **before** the workload starts

Incident Response & Pipeline Failures

- In DevSecOps, a pipeline failure due to security is **good news**:
  - Security issue is caught **before production**
- Important to have a basic **incident response** process when:
  - SCA finds critical vulnerabilities
  - Image scan detects high-severity OS issues
  - Policy-as-Code blocks deployment

Example: Handling a Critical Vulnerability in CI

1. Pipeline fails because SCA found a critical vulnerability.
2. CI system creates a ticket in the issue tracker automatically.
3. Team actions:
    - Assign an owner
    - Evaluate impact and exploitability
    - Prioritize patch (e.g., hotfix release)
4. Communication:
    - Inform stakeholders if production is at risk
    - Document decisions in the ticket

Example: Gatekeeper Blocking Deployment

- Deployment job error:

```
Error from server (Forbidden): error when creating "deployment.yml":
admission webhook "validation.gatekeeper.sh" denied the request:
Containers must set securityContext.runAsNonRoot = true
```

- Response:
  - Developer updates deployment YAML:

```yaml
securityContext:
  runAsNonRoot: true
  runAsUser: 1000
```

- Re-applies manifest
- Deployment succeeds once policy is satisfied

Building a Secure Pipeline Step by Step

- Start simple:
  - Enable **dependency scanning** (SCA) on PRs
  - Add **container image scanning** for main branches
- Then:
  - Introduce **IaC scanning** for Terraform/Kubernetes
  - Move secrets into **secret managers** or CI secret stores
- Finally:
  - Enforce **policy-as-code** gates for production deployments
  - Define and rehearse **incident response** playbooks

DevSecOps Mindset for Students

- Think of security as a **quality attribute**, not an afterthought
- When you design a pipeline, ask:
  - Where can things go wrong?
  - How can we detect issues earlier?
  - How can we make secure behavior the **default**?
- Your future backend/DevOps work will likely require:
  - Understanding these tools
  - Integrating them in real-world CI/CD systems

Summary & Key Takeaways

- DevSecOps integrates security into every step of the SDLC
- **Shift left**: detect issues early, fix them cheaply
- Use automated tools for:
  - **Dependency scanning**
  - **Image scanning**
  - **IaC & configuration scanning**
- Manage secrets safely and harden IAM roles
- Enforce rules with **policy-as-code** and tools like OPA Gatekeeper
- Treat pipeline security as a **core responsibility** of backend and DevOps engineers